

4교시: Compose 네트워크와 service name

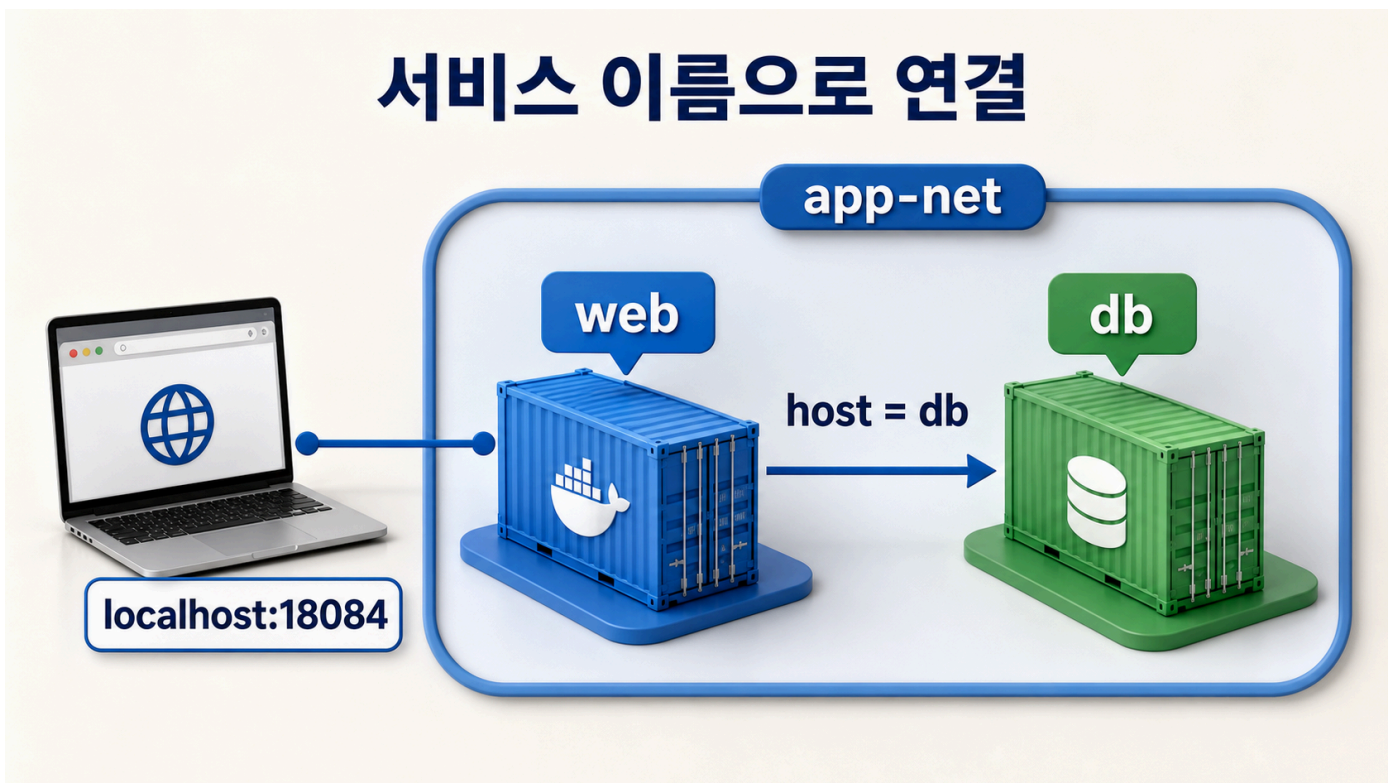
수업 목표

- Compose project network 안에서 service name이 DNS 이름처럼 동작함을 설명한다.
- host의 localhost 와 container network의 db 이름을 구분한다.
- depends_on 과 healthcheck 의 역할과 한계를 설명한다.

50분 흐름

시간	활동	비중	산출
0-10분	Day 3 custom network 복습	설명 20%	network 비교
10-22분	Compose default/custom network 설명	설명 25%	service DNS note
22-35분	db-client로 service name 확인	실행 30%	DNS evidence
35-45분	depends_on과 healthcheck 토론	설명 15%	readiness note
45-50분	Week 3 MSA 연결	설명 10%	next concept

Visual 1: service name DNS



이 visual은 같은 Compose network 안의 container가 db 라는 service name으로 PostgreSQL service에 접근하는 흐름을 보여준다.

실행 명령

```
cd week2/day4/labs/compose-app
docker compose up -d
docker compose run --rm db-client pg_isready -h db -U paperclip -d paperclip
docker compose ps
```

기대 결과:

```
db:5432 - accepting connections
```

핵심 설명

Compose는 project마다 network를 만든다. 명시한 network가 없으면 default network를 만들고, 이 network 안의 service는 service name으로 서로를 찾을 수 있다. Day 4 실습은 app-net 을 명시해 web, db, db-client가 같은 network에 들어가도록 한다.

중요한 구분은 host와 container network다. host terminal에서 curl http://localhost:18084 는 host port를 통해 web에 접근한다. 반면 db-client container 안에서 -h db 는 Compose network의 service name을 사용한다. host terminal에서 db 라는 이름이 자동으로 풀리는 것은 아니다.

depends_on 은 service 시작 관계를 표현한다. 여기에 condition: service_healthy 를 사용하면 db healthcheck가 healthy가 된 뒤 의존 service를 시작하도록 도울 수 있다. 하지만 application-level readiness 전체를 보장하는 만능 장치는 아니다. 실제 app은 연결 재시도, timeout, error handling을 가져야 한다.

판단 기준

오해	바로잡기
container IP를 README에 기록	service name을 기록한다
host에서 db:5432 접속 기대	같은 Compose network의 container에서 사용한다
depends_on이면 DB query 준비 완료	healthcheck와 앱 재시도를 함께 본다
default network면 항상 충분	격리나 명시성이 필요하면 named network를 둔다

기록 템플릿

Lesson 4 Network Evidence

- Compose project:
- Network:
- DB service name:
- Client command:
- Result:
- host localhost와 service name 차이:
- depends_on 한계:

평가 기준

기준	2점 evidence
service name	db로 접속한 결과를 기록했다
경계 구분	host localhost와 container DNS를 구분했다
readiness	depends_on과 healthcheck 한계를 설명했다

공식 근거 링크

- Networking in Compose: <https://docs.docker.com/compose/how-tos/networking/>
- Compose networks reference: <https://docs.docker.com/reference/compose-file/networks/>

선행 지식과 범위 경계

학생은 Day 3에서 custom bridge network를 만들고 container name으로 통신했다. Day 4에서는 Compose가 project network를 만들고 service name 기반 discovery를 제공한다는 점을 배운다.

이 교시는 DNS 내부 동작 전체를 깊게 다루지 않는다. 핵심은 host의 `localhost`, container 내부의 `localhost`, Compose network의 service name을 구분하는 것이다.

학술 기준 연결

네트워크 이름 해석은 추상화의 좋은 사례다. 학생은 IP address라는 구현 detail보다 service name이라는 안정적인 interface를 사용한다. 이는 distributed systems에서 location transparency와 service discovery의 기초가 된다.

Bloom taxonomy로 보면 이 교시는 "구분", "분석", "정당화" 단계다. 학생은 왜 container IP를 README에 쓰지 말아야 하는지 설명하고, service name을 쓰는 이유를 정당화해야 한다.

세 가지 주소 공간

초급자가 가장 많이 혼동하는 것은 `localhost` 다. 다음 표를 반복해서 확인한다.

위치	<code>localhost</code> 의미	db 의미
host terminal	host 자신	보통 해석 안 됨
web container 내부	web container 자신	같은 Compose network의 db service
db container 내부	db container 자신	자기 service name 또는 network alias

따라서 host browser는 `http://localhost:18084` 로 web에 접근하고, container 간 DB 연결은 `db:5432` 를 사용한다.

service name을 쓰는 이유

container IP는 lifecycle에 따라 바뀔 수 있다. container를 삭제하고 다시 만들면 IP가 달라질 수 있다. service name은 Compose file에 남는 의도된 이름이다.

실무 handoff에서는 다음처럼 기록한다.

Application container에서 DB host는 db를 사용한다.
Host machine에서 DB에 직접 접근하려면 별도 ports publish가 필요하다.

이 문장에는 중요한 경계가 있다. `db` 는 host machine의 DNS 이름이 아니라 Compose network 내부 이름이다.

depends_on 과 readiness

`depends_on` 은 service 사이의 관계를 표현한다. 하지만 관계에는 여러 수준이 있다.

수준	의미	예
create order	먼저 container를 만든다	db container 생성 후 web 생성
start order	먼저 start한다	db start 후 web start
health condition	healthcheck 성공을 기다린다	<code>service_healthy</code>
application readiness	실제 요청 처리 가능	app retry와 query 성공

Compose의 `depends_on.condition: service_healthy` 는 유용하지만 마지막 수준까지 보장하지 않는다. application code가 DB 연결 retry를 하지 않으면 짧은 순간의 지연에도 실패할 수 있다.

실습 확장: network inspect

아래 명령은 선택 실습으로 사용한다.

```
docker network ls --filter name=compose-app
docker network inspect compose-app_app-net
```

확인할 항목:

- network 이름
- 연결된 container 이름
- service container의 IP
- 같은 network에 web과 db가 모두 있는지

단, README에는 IP를 정상 연결 정보로 기록하지 않는다. IP는 관찰 evidence일 뿐, handoff 계약은 service name이다.

실무 failure mode

Failure mode	증상	원인 후보	확인
host에서 db 접속	name resolution 실패	network boundary 오해	host/container 위치 확인
app에서 localhost:5432 사용	DB 연결 실패	app container 자신을 바라봄	connection string
service 이름 오타	DNS 실패	db vs postgres 혼동	Compose service name
depends_on 과신	startup race	readiness와 retry 부족	logs, healthcheck

오해 점검 문항

문항	기대 답
web container에서 DB host를 localhost로 쓰는가	아니다. localhost는 web 자신이다
host browser가 http://web으로 접근하는가	아니다. host port publish를 사용한다
DB container IP를 README에 기록해야 하는가	아니다. service name을 기록한다
healthcheck가 있으면 app retry가 필요 없는가	아니다. 둘은 다른 계층이다

NIST NICE 관점의 보안 연결

DB service name과 network boundary는 보안 책임과도 연결된다. DB port를 host에 publish하지 않으면 host 외부에서 직접 접근하는 경로가 줄어든다. 물론 이것만으로 보안이 완성되지는 않지만, 노출면을 줄이는 기본 습관이다.

Day 4 실습은 DB에 ports 를 열지 않는다. 학생은 "왜 web은 host port를 열고 DB는 열지 않는가"를 설명해야 한다.

전이 과제

Week 3 MSA를 예고하며 다음 질문을 작성한다.

질문	답
frontend가 api를 부를 때 host 이름은 무엇이어야 하는가	
api가 db를 부를 때 host 이름은 무엇이어야 하는가	
host browser에서 접근해야 하는 service는 무엇인가	
외부에 공개하면 안 되는 service는 무엇인가	

이 전이 과제는 Compose service name을 MSA service communication으로 확장한다.

실무 설계 예시: DB port를 열지 않는 이유

Day 4 실습에서는 web service만 host port를 publish한다. DB service는 같은 Compose network 안의 service가 접근하도록 두고, host에는 직접 열지 않는다.

이 설계는 다음 판단을 담고 있다.

판단	설명
필요한 경로만 공개	browser가 필요한 것은 web 접근이다
dependency는 내부 통신	web이 DB를 service name으로 호출한다
노출면 감소	host 또는 외부 사용자가 DB port에 직접 접근하지 않는다
실습 단순화	DB 확인은 db-client container로 수행한다

운영 환경에서는 network policy, firewall, secret manager, managed database access rule이 추가로 필요하다. 하지만 초급 단계에서는 "열 필요가 없는 port는 열지 않는다"는 원칙을 먼저 익힌다.

캡처 가이드

수업 중 screenshot을 남길 경우 다음 화면을 추천한다.

screenshot	포함할 내용
compose-network-ps.png	docker compose ps의 web/db 상태
compose-db-client.png	db-client가 db host로 접속한 결과
compose-network-diagram-note.png	host localhost와 service name db 차이 메모

screenshot은 장식이 아니라 evidence다. 파일명에는 날짜보다 실습 의미가 드러나는 이름을 사용한다.

평가자 질문

평가자는 학생에게 다음 질문을 던질 수 있다.

1. host browser는 왜 localhost:18084 를 쓰는가?
2. db-client 는 왜 db 라는 host 이름을 쓸 수 있는가?
3. DB port를 host에 열지 않아도 DB 확인이 가능한 이유는 무엇인가?
4. 이 구조가 Week 3 MSA의 service communication과 어떻게 이어지는가?

네 질문에 모두 답할 수 있으면 service name과 network boundary를 실무적으로 이해한 것이다.